

A Nearest Neighbors based Data Filter for Fog Computing in IoT Smart Agriculture

Franklin Magalhães Ribeiro Jr.
Federal University of ABC
Federal Institute of Maranhão
franklin.junior@ufabc.edu.br

Ronaldo Prati
Federal University of ABC
Santo André, Brazil
ronaldo.prati@ufabc.edu.br

Reinaldo Bianchi
Centro Universitário FEI
São Bernardo do Campo, Brazil
rbianchi@fei.edu.br

Carlos Kamienski
Federal University of ABC
Santo André, Brazil
carlos.kamienski@ufabc.edu.br

Abstract— In smart agriculture, the Internet of Things (IoT) makes it possible to analyze and manage agricultural yield to increase productivity, reduce wasted resources, and decrease irrigation costs. In IoT systems, if data management is entirely performed in the cloud, the system may not work correctly due to connectivity problems, which is common in some remote regions where the agribusiness thrives. A fog computing solution enables the IoT system to process data faster and deal with intermittent connectivity. However, a high number of packets sent from the fog to the cloud can cause link congestion with mostly useless data traffic. Dealing with fog data filtering is a challenge because it requires knowing which data is essential to send to the cloud. This paper proposes an approach to collect and store data in a smart agriculture environment and two different methods filtering data in the fog. We designed an experiment for each filtering method, using a real dataset containing temperature and humidity values. In both experiments, the fog filters the data using the k-Nearest-Neighbors (kNN) algorithm, which classifies data into categories according to their value ranges. In the first experiment, the fog classifies the data and generates an output of the number of data categories. In the second experiment, data is classified and also compressed based on the previously obtained categories using the run-length encoding (RLE) technique to preserve the data time series nature. Our results show that data filtering reduces the amount of data sent by the fog to the cloud.

Keywords—smart agriculture, fog computing, Internet of things, data filtering, machine learning.

I. INTRODUCTION

There are various challenges to be overcome in modern agriculture, such as pest control, production costs, and irrigation optimization. One way to manage agriculture resources is through the concept of smart agriculture, which aims to integrate services in rural areas and to use their resources intelligently through IoT technologies [1], [2]. In the agriculture context, it is possible to use smart applications to avoid wastes and to manage resources. For example, to make an irrigation decision [3], an application could perceive relevant data using IoT, like weather conditions, soil moisture, or crop growth stage.

IoT systems can handle an enormous amount of data through physical devices (sensors and actuators) transmitting and receiving data in an application context [4]. However, most of these systems only process the collected data on a central server.

Fog computing allows data analyses locally at the edge [5], reducing the need for decisions making by an external server (connected through the Internet) and reducing the need to send data to a cloud [6]. Fog computing can store, analyze, and filter the data received by the sensors without dependency on the constant cloud connection availability [6]. Additionally, using fog computing, the actuator can

execute a decision near real-time [7], [8], [5]. Then, in a smart irrigation context, based on data collected by sensors, the fog can decide when, where, and how much water is needed to irrigate a crop and the system will perform this action immediately. Fog computing can reduce network data traffic [7], [8]. However, it is challenging to manage fog data flow, considering scalability, security, data storage capacity, processing power, and battery limitations [9].

In this paper, we propose a fog computing approach to collect, filter, store, and transmit data in a smart farming application, using two filtering methods in the fog. In the both filtering methods, we propose to use a well-known supervised learning algorithm, the k-Nearest-Neighbors (kNN) [10], like a fog data filter, to reduce the amount of data stored in the fog/cloud and optimize the data channel throughput between fog and cloud communication channel. We propose the use of two fog-based data filter methods and design an experiment to evaluate each filtering method. We discovered that, in smart agriculture, the fog should be capable of learning with the farm environment to know when the filter should be applied.

In the first experiment, we used a filtering method that executes the kNN technique, which reduced approximately 97% of the data the fog needs to transfer to the cloud. We also observed that the fog should split the filter period to know about the data behavior when the data have many variations. In the second experiment, we used a filtering method that executes the combined kNN and the run-length encoding (RLE) techniques [17]. This method reduces the data size by approximately 98% and clearly maintains the time-series nature for different data categories.

This paper is organized as follows: In Section 2 we describe the literature background and related work, Section 3 details our approach to smart agriculture, Section 4 describes fog data filter experiments, Section 5 reports the results discussed in Section 6. Finally, Section 7 presents conclusions and future work.

II. BACKGROUND AND RELATED WORK

A. Fog Computing

Fog computing could deal with the challenge of massive data analysis in real-time on the edge [11]. With fog computing, IoT systems may perform faster data analysis, a significant advantage for latency-sensitive systems.

A hierarchical Fog-based IoT architecture includes three stages: thing, fog, and cloud [6], [12]. The thing stage consists of embedded systems like sensors, actuators, and devices that allow users to connect to fog. The fog stage consists of physical machines, switches, routers, and access points, where the collected data can be processed. The cloud stage is connected to the fog stage (fog) by a gateway over the Internet.

B. kNN Algorithm

The k-Nearest-Neighbors (kNN) algorithm [10] is a machine learning algorithm used to classify new data samples into categories. It computes the Euclidean distance between the values of a new data sample with other previously classified samples to find the k samples closer to the new data sample. Thus, the algorithm classifies the new data samples, according to the most often k nearest neighbors. The underlying assumption is that objects close to each other have a higher probability of belonging to the same category. Therefore, this estimation allows us to identify the category to which the new data sample belongs. As in smart agriculture, the values collected by sensors can be redundant. Using the kNN will allow the fog to maintain a smaller number of data in the Fog database.

Here we choose the kNN algorithm because its computational cost is small and compatible with resource-constrained IoT devices, unlike recent algorithms such as deep neural networks. Furthermore, it has no additional computing cost to learn the training data set during the fog filter execution. However, it is possible to use other machine learning classification algorithms, like decision trees or random forest, since any other algorithms from this kind can classify the data.

C. Run-Length Encoding (RLE)

Run-length encoding (RLE) is a data compression technique that preserves the order of data values [17], which makes it possible to recover the data by the original order. In other words, when we apply RLE in a data time series, it is possible to recover the order of the events that occurred in the time series.

D. Related Work

The investigation in [8] showed that system time to respond drops 73% when fog analyses data locally against a cloud. The fog could also reduce the cloud's sending data by 93%, as shown in research by [7]. However, these studies do not lead to real scenarios and devices scalability, since in real applications, IoT deals with thousands of sensors. Though a fog-based data filter can obtain gains, it is still challenging to find a solution that can cover different IoT scenarios, such as a farm irrigation scenario. Therefore, dealing with fog data-level communication is a research challenge [13].

III. FOG DATA FLOW

Our proposal involves adopting a fog computing approach in smart agriculture, where sensors are responsible for collecting the data and a fog node for data storage, analysis, filtering, and the transmission of the filtered data to the cloud (Fig. 1). In our proposal, the fog receives data from the sensors and filters it by choosing to execute two different filtering methods using a data classification technique, to reducing data size, and consequently, the data traffic between fog and cloud.

The data flow works as follows: the sensors collect data from a farm environment and send it to the fog. The data arrives in the fog through a network server, transferring the raw data to the data storage module and the data analytics module. The data analytics module analyses the raw data to make decisions, then, it stores the decisions in fog memory and sends the decisions to actuators to irrigate a crop. The data storage module is responsible for storing data in memory. It can provide past decisions and filtered data to the analytics module. Furthermore, the data storage module

provides the raw data periodically to the fog data filter. After a specific time period, the fog filters the (old) raw data stored, classifies it, deletes this (old) raw data from memory and sends the classified data to the cloud via the MQTT protocol. Thus, the cloud may need (in the future) to perform an analysis with historical classified data. Therefore, the system operates processing the data locally, without any cloud action.

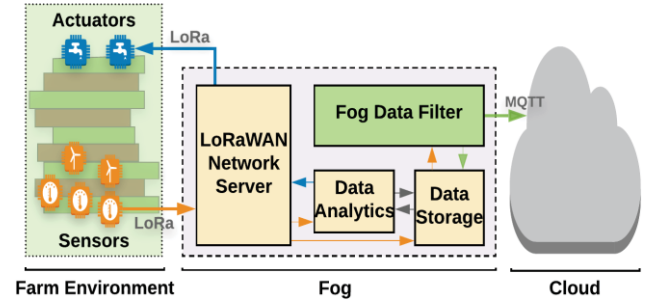


Fig. 1. Conceptual Fog Data Flow to Smart Agriculture.

In a real farm scenario, the data may be sent by sensors via LoRaWAN, whose packets contain values such as temperature and humidity. On the other hand, the fog typically hosts different software modules, such as data analytics, data storage, data filtering, and the LoRaWAN backend server, such as ChirpStack¹ (Fig. 1). The use of LoRaWAN in smart agriculture has many advantages, such as providing security to the data communication channel, extending sensor battery lifetime, and providing a long-range distance in the transmission between the sensors and the fog [1], [2].

The fog data filter can operate by two methods. The choice between the two fog data filter methods is a design choice. The first method is responsible for the data classification using the kNN and for quantifying the number of data in a category and sending those accounted categories to the cloud. The second method uses the kNN and the run-length encoding (RLE) techniques to compress each classified data register by the order, to preserve the time-series nature of the data categories.

The fog filter runs at intervals initially defined by the application developer, but the fog may automatically configure this interval depending on the context or on abrupt variations in the data patterns. With this approach, we expect to reduce the amount of raw.

The fog data filter has three advantages, namely (i) creating a new dataset for future decision-making based on the characteristics of the categories, (ii) decreasing the data traffic between fog and cloud, and (iii) reducing the data storage requirements in fog and cloud.

IV. RESEARCH DESIGN AND METHODS

A. Training Data Set

For dealing with smart agriculture, we used the data categories reported by the review work in [15] as a training dataset, which specifies the temperature and humidity best conditions for tomatoes cultivation. Shamshiri et al., define the weather conditions for tomatoes crop as lower optimal, upper optimal, lower failure, upper failure, lower marginal, upper marginal, and optimal. Additionally, we defined critical dry and cold and humid categories because some value range conditions are not defined by [15].

We generated the training dataset² with 962 registers, using a rule with temperature intervals of 1°C, and humidity

1. <https://www.chirpstack.io/>

2. <https://github.com/frank20192020/metroagriforTrainingDataSet>

intervals of 5 in 5%. We summarize this rule into general intervals in Table 1.

TABLE I. TRAINING DATASET

Class	Category	Range between values	
		Temperature (°C)	Humidity (%)
0	Critical dry	0-47	5-35
1	Lower fail	0-11	40-65
2	Marginal	12-28	40-55
7	Upper Fail	29-47	40-55
8	Cold and Humid	0-11	70-100
2	Marginal	12-14	60-100
3	Lower optimal	15-17	60-100
4	Optimal	18-27	60-100
5	Upper Optimal	28-31	60-100
6	Upper Marginal	32-35	60-100
7	Upper Fail	36-47	60-100

We also calculated the ROC area to evaluate the data training model performance. Then, we plotted the ROC curves for each class (category) in the training dataset (Table 1). On average, the ROC returned a performance of 89% for the data-training model (Fig. 2).

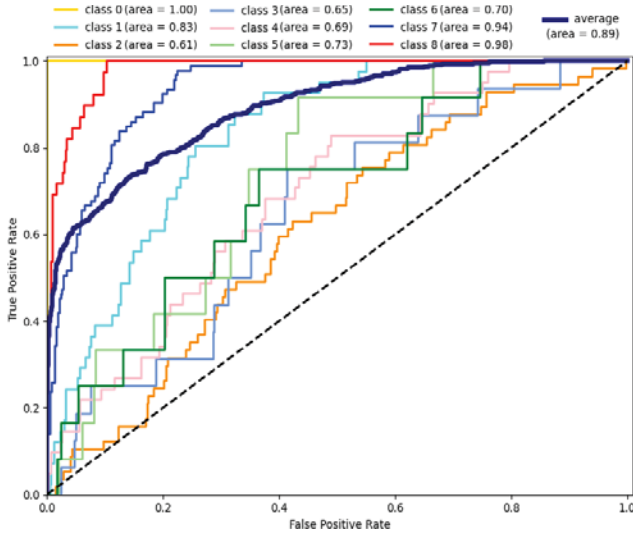


Fig. 2. ROC curves for each class (category) to the training dataset.

B. Input (Test) Data Set

Since our scope is the data filtering, we assume that the data is already available in the fog node. We used a dataset collected in August 2013 near the *Cuerda del Pozo* reservoir, in Soria, Spain [14], with 44,026 temperature and humidity records.

C. Experiment Design and Execution

Both fog filtering methods perform kNN to categorize the data of an original data set [14] according to the training data set. We used the Python programming language to implement the fog data filter module, where for the kNN we used $k=3$. We divided the classification process into two rounds: for the round 1, we classified the data collected between 00:00 and 06:00 and for the round 2, between 06:01 and 12:00 (both on the first day of August 2013). We choose this time interval because we consider that the cloud receives a classified data file every 6 hours in a real scenario.

We conducted experiments with the fog filter, using a real dataset [14], and training the dataset (Table 1) based in [15]. We executed the round 1 and round 2 data time series in experiments for each filtering method.

Both filtering methods perform the following steps: (i) read temperature and humidity records, (ii) normalize data

on a scale of 0 to 1 (where 0 and 1 correspond to the lowest and highest value in the training dataset, respectively), (iii) calculate the Euclidean distance to classify the records and (iv) generate an output with categorized data.

In the first experiment, after the data classification, the filter counts the number of occurrences for each data category. On the other hand, in the second experiment, the filter performs the run-length encoding technique that counts and joins same consecutive data categories, during the kNN classification. We also generated graphical plots of input data and output data.

V. RESULTS

After executing the kNN algorithm in two rounds, we obtained the categorized data, generating two graphic plots for each round: (i) a contour plot for the input dataset and (ii) a two-dimensional correlogram, with the data categories distribution. To generate the correlogram and the contour plot, we used the *seaborn* library in the Python programming language.

A. Input (Test) Dataset Density Distribution

Fig. 3 depicts two contour curve plots to visualize the input data's behavior, showing the density distribution of all temperature and humidity records for rounds 1 and 2. The contour curve plots (Fig. 3) are on the same scale and allow us to visualize the highest data density according to color tone (in dark blue). Therefore, the value ranges with the darker coloring reveal the most frequent episodes.

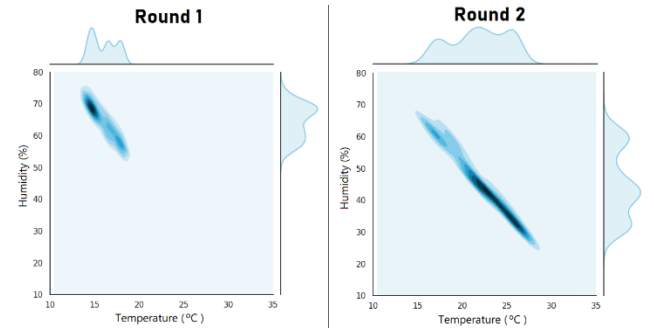


Fig. 3. Contour curve plot of input data (to round 1 and round 2).

For round 1, the lowest and highest temperature values were 14.1°C and 18.4°C, respectively, while the lowest and highest humidity values were 53.4% and 75.4%, respectively, with the standard deviation of the temperature equals to 1.1375 and the standard deviation of humidity equals to 5.3143. For round 2, the lowest and highest temperature values were 15.1°C and 28.6°C, respectively, while the lowest and highest humidity values were 23.4% and 68.69%, respectively, with the standard deviation of the temperature equals to 4.11 and the standard deviation of humidity equals to 13.2867.

In round 1, the average temperature was 16.01°C, and the average humidity was 64.43%. In round 2, these averages were 22.44°C and 43.76%. Also, Fig. 2 reveals a higher value dispersion in round 2 data input, which it is more evident when we observe the standard deviation of temperature and humidity for round 2.

B. Experiment 1: Data Classification

After executing the first filtering method, it generated an output file (in CSV format) containing the data categories and the number of data classified in that category (Table 2). This output file may then be sent from fog to cloud, instead of the original raw data.

Note that there are originally 721 temperature and humidity records in the dataset, 361 in round 1, and 360 in round 2. By filtering, we reduced the data to a table with nine classes and the respective number of occurrences for each class (Table 2). Therefore, we always have as output a file with nine categories and nine numbers of occurrences for each data category.

In round 1, the original input data has 5277 bytes, and the output file has 149 bytes. In round 2, the original input data has 5381 bytes, and the output file has 151 bytes. Then, there was a reduction in the total data volume of approximately 97% for both rounds in this context.

TABLE II. FILTERED DATA OUTPUT (TO ROUND 1 AND ROUND 2)

Class	Category	Round 1	Round 2
0	Critical dry	0	136
1	Lower fail	0	0
2	Marginal	61	89
3	Lower optimal	152	12
4	Optimal	148	19
5	Upper Optimal	0	0
6	Upper Marginal	0	0
7	Upper Fail	0	23
8	Cold and Humid	0	81

We observed that categories marginal, lower optimal, and optimal correspond to 100% of the input data in round 1, and categories critical dry, marginal, and cold and humid correspond to 85% of the input data in round 2 (Table 2). This behavior happens because the input data values from rounds 1 and 2 occur more frequently in the temperature and humidity intervals covered by these categories in the training dataset (Fig. 3).

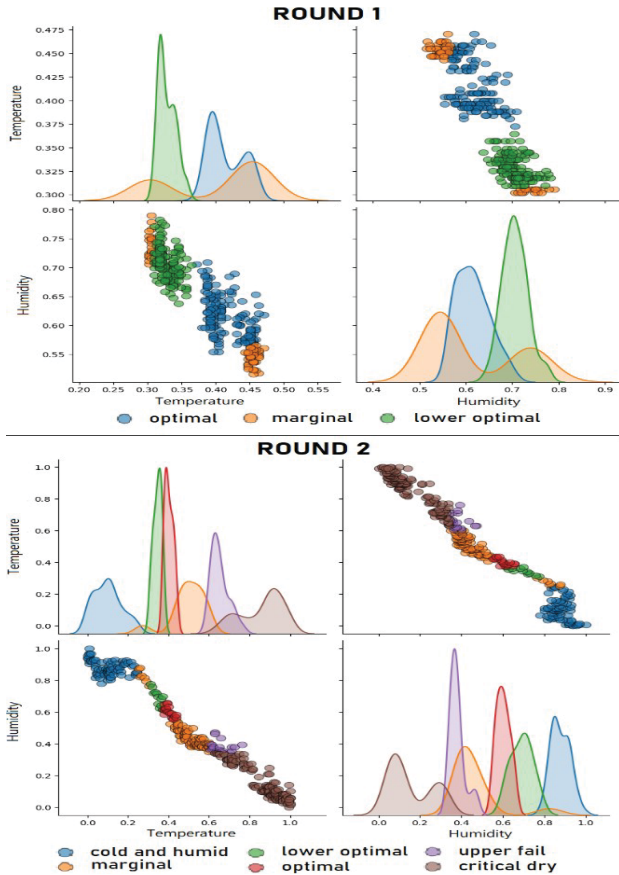


Fig. 4. Correlogram of round 1 and 2 categories (different scales)

Fig 4 shows a correlogram for each round comprising four graphic plots. The correlogram includes two density plots and two dispersion plots, highlighting the category

density according to temperature and humidity. In some density curves, we observed that classes overlap, although it does not prevent data classification from occurring because we deal with two attributes. The overlap in the humidity or temperature curves happens because many training data categories cover the same ranges (Table 1).

C. Experiment 2: RLE and Data Classification

The second filtering method performs kNN and RLE. In a data time series, the kNN produces a category for each data record. The RLE counts the quantity of the same categories that appear consecutively and then generates an output containing the category name and the number of consecutive occurrences for that category. We obtained the output of this filtering method for round 1 and round 2 in Table 3. After it, the fog concatenates each output line into a string, according to the categories' occurrence order, keeping the time-series nature of the data.

TABLE III. FILTERED DATA OUTPUT ACCORDING TO THE ORDER OF DATA TIME SERIES (TO ROUND 1 AND ROUND 2)

Round 1			Round 2		
Category	Q	Out	Category	Q	Out
optimal	1	1op	cold and humid	81	81ch
marginal	3	3ma	marginal	7	7ma
optimal	15	15op	lower optimal	12	12lo
marginal	3	3ma	optimal	11	11op
optimal	1	1op	marginal	1	1ma
marginal	4	4ma	optimal	6	6op
optimal	1	1op	marginal	2	2ma
marginal	1	1ma	optimal	2	2op
optimal	7	7op	marginal	79	79ma
marginal	2	2ma	upper fail	8	8uf
optimal	1	1op	critical dry	1	1cd
marginal	4	4ma	upper fail	3	3uf
optimal	1	1op	critical dry	1	1cd
marginal	2	2ma	upper fail	1	1uf
optimal	3	3op	critical dry	1	1cd
marginal	4	4ma	upper fail	3	3uf
optimal	1	1op	critical dry	2	2cd
marginal	4	4ma	upper fail	1	1uf
optimal	3	3op	critical dry	1	1cd
marginal	2	2ma	upper fail	2	2uf
optimal	6	6op	critical dry	9	9cd
marginal	3	3ma	upper fail	2	2uf
optimal	1	1op	critical dry	9	9cd
marginal	4	4ma	upper fail	1	1uf
optimal	2	2op	critical dry	3	3cd
marginal	4	4ma	upper fail	1	1uf
optimal	10	105op	critical dry	4	4cd
lower optimal	72	72lo	upper fail	1	1uf
marginal	12	12ma	critical dry	105	105cd
lower optimal	9	9lo			
marginal	9	9ma			
lower optimal	71	71lo			

Q = quantity; Out = output data according to data time series

In round 1, the output is a text file containing the string: “1op3ma15op3ma1op4ma1op1ma7op2ma1op4ma1op2ma3op4ma1op4ma3op2ma6op3ma1op4ma2op4ma105op72lo12ma9lo9ma71lo”, as the same output order in Table 3.

In round 2, the output is a text file containing the string: “81ch7ma12lo11op1ma6op2ma2op79ma8uf1cd3uf1cd1uf1cd3uf2cd1uf1cd2uf9cd2uf9cd1uf3cd1uf4cd1uf105cd”, as the same output order shown in Table 3.

Note that round 1 has 361 registers with 5277 bytes, and the round 2 has 360 registers with 5381 bytes. The filtering

using kNN and RLE reduced the data size for round 1 to 102 bytes and for the round 2 to 93 bytes, i.e., a reduction of 98,06% and 98,27%, respectively.

VI. DISCUSSION

We perceived that in round 1, the input data had close values and categories due to the thermal amplitude controlled by relative humidity. However, this did not occur in round 2, as the data had a high thermal amplitude of more than 10 degrees Celsius, which is expected in the Soria region (Spain) in August.

In experiment 1, we observed that, instead of waiting 6 hours or a defined period to send the data to the cloud, the fog data filter should automatically adapt to the weather situation or any context circumstances. If the cloud receives data with abrupt values as in round 2, there will be no knowledge about the historical data behavior in the future. For example, the cloud will not know the weather variations that occurred in this period. Therefore, to not risk future analysis of historical data stored in the cloud, we observed evidence in this investigation, suggesting that the IoT system to smart agriculture should have an adaptive mechanism to learn from the context and decide the optimal time interval to perform the data filtering periodically. Consequently, the fog based IoT system should learn by the agricultural context.

In experiment 2, we observed that the filter reduced the data size because the RLE joined many consecutive repeated categories. However, supposing that the input only has different consecutive categories, the output size will be close to the original input data size, but, differently from the first method, the fog filter using RLE always maintains the time-series nature of the data. So, there is a trade-off between choosing fog data filter methods.

VII. CONCLUSION AND FUTURE WORK

In this paper, we presented an approach to collect, store, analyze, and filter data in a smart agriculture scenario. We also proposed a solution to filter the data in the fog using the k-Nearest-Neighbors (kNN) algorithm. We analyzed the fog data filter considering a smart irrigation context, using a real database, exploring the attributes (dimensions) of temperature and humidity [14]. For the filtering context, we considered a tomato crop used as training data [15]. In this work, the experiments represented a proof of concept because the training data samples are limited since we generated it artificially based on [15].

Our first and second fog filter methods reduced the size of the data stored in the fog, respectively, by 97% and 98%, and consequently, the future data traffic between fog and cloud. For the first fog filter method, the data volume is equivalent to the number of categories (multiplied by two), and the problem domain defines the number of categories. However, the data volume of the second filter method depends on the data time series classification. For using the fog data filter, the problem domain in the smart agriculture application must be well defined. Contexts with many dimensions can increase algorithm operations and demand high CPU power on fog computing. Therefore, the system designers need to determine which attributes are relevant to collect by the sensors in the smart agriculture context.

In the investigation, we also realized that the filtering time and interval must be automatically defined by the smart agriculture system, whose system must learn the context circumstances to perform the filter. In other words,

the smart agriculture system should adapt the filter itself based on the sensors collected data from the farm environment. Finally, as future work, we will implement the adaptive fog data filter capable of learning by the crop context. We also intend to replicate the experiment in an operational IoT system, using a case study of SWAMP pilots [16], which provides a hardware infrastructure to a real smart farming scenario.

ACKNOWLEDGMENTS

This research has been jointly funded by the European Commission in Europe and MCTIC/RNP in Brazil, under the EUB-02-2017 IoT Pilots call. It is also supported by the Federal Institute of Education, Science and Technology of Maranhão (IFMA).

REFERENCES

- [1] O. Elijah, T. A. Rahman, I. Orikumhi, C. Y. Leow and M. N. Hindia, "An Overview of Internet of Things (IoT) and Data Analytics in Agriculture: Benefits and Challenges," in *IEEE Internet of Things Journal*, vol. 5, no. 5, pp. 3758-3773, Oct. 2018.
- [2] M. S. Mekala and P. Viswanathan, "A Survey: Smart agriculture IoT with cloud computing," 2017 International conference on Microelectronic Devices, Circuits and Systems (ICMDCS), 2017.
- [3] L. García, L. Parra, J.M. Jimenez, J. Lloret, and P. Lorenz. IoT-Based Smart Irrigation Systems: An Overview on the Recent Trends on Sensors and IoT Systems for Irrigation in Precision Agriculture. *Sensors* 2020, 20, 1042.
- [4] L. Atzori, A. Iera, and G. Morabito, "Understanding the Internet of Things: definition, potentials, and societal role of a fast evolving paradigm", *Ad Hoc Networks*, 56, pp.122-140, March 2017.
- [5] C. Mouradian, D. Naboulsi, S. Yangui, R. H. Glitho, M. J. Morrow, and P. A. Polakos. A comprehensive survey on fog computing: State-of-the-art and research challenges. *IEEE Communications Surveys Tutorials*, 20(1):416-464, Firstquarter 2018.
- [6] M. Chiang and T. Zhang. Fog and iot: An overview of research opportunities. *IEEE Internet of Things Journal*, Dec 2016.
- [7] T. N. Gia, M. Jiang, A. Rahmani, T. Westerlund, P. Liljeberg, and H. Tenhunen. Fog computing in healthcare internet of things: A case study on ecg feature extraction. *IEEE CIT/IUCC/DASC/PICOM* 2015, pages 356-363, 2016.
- [8] J. Li, J. Jin, D. Yuan, M. Palaniswami, and K. Moessner. Ehopos: Data-centered fog platform for smart living. *International Telecommunication Networks and Applications Conference*, 2015.
- [9] A. Yousefpour, C. Fung, T. Nguyen, K. Kadiyala, F. Jalali, A. Niakanlahiji, J. Kong, and J. P. Jue, "All one needs to know about fog computing and related edge computing paradigms: A complete survey", *Journal of Systems Architecture*, Volume 98, 2019.
- [10] J. Han, M. Kamber, J. Pei. Data mining: concepts and techniques. Morgan Kaufmann Publishers, 2011. ISBN 978-0-12-381479-1.
- [11] S. Verma, Y. Kawamoto, Z. M. Fadlullah, H. Nishiyama, and N. Kato. A survey on network methodologies for real-time analytics of massive iot data and open research issues. *IEEE Communications Surveys Tutorials*, 19(3):1457-1477, thirdquarter 2017.
- [12] M. Mukherjee, L. Shu, and D. Wang. Survey of fog computing: Fundamental, network applications, and research challenges. *IEEE Communications Surveys Tutorials*, 20(3):18261857, 2018.
- [13] H. Atlam, R. Walters, and G. Wills, "Fog Computing and the Internet of Things: A Review". *Big Data Cogn. Comput.* 2018.
- [14] F. Aguilar, "Press, Temperature and Humidity August 2013", *DataONE Dash, Dataset*, doi.org/10.15146/R3730R, January 2014.
- [15] R. Shamshiri, J. W. Jones, K. R. Thorp, D. Ahmad, H. Man, and S. Taheri. Review of optimum temperature, humidity, and vapour pressure deficit for microclimate evaluation and control in greenhouse cultivation of tomato: a review, *International Agrophysics*, 32(2), 287-302, 2018.
- [16] C. Kamiński, J. Soininen, M. Taumberger, S. Fernandes, A. Toscano, T. Salmon, R. Filev, and A. Torre. Swamp: an iot-based smart water management platform for precision irrigation in agriculture. *Global IoT Summit 2018 (GloTS'18)*, June 2018.
- [17] D. Salomon. Data Compression. The Complete Reference 4th edition. Springer-Verlag London Limited. ISBN 978-1-84628-602-5, 2007.